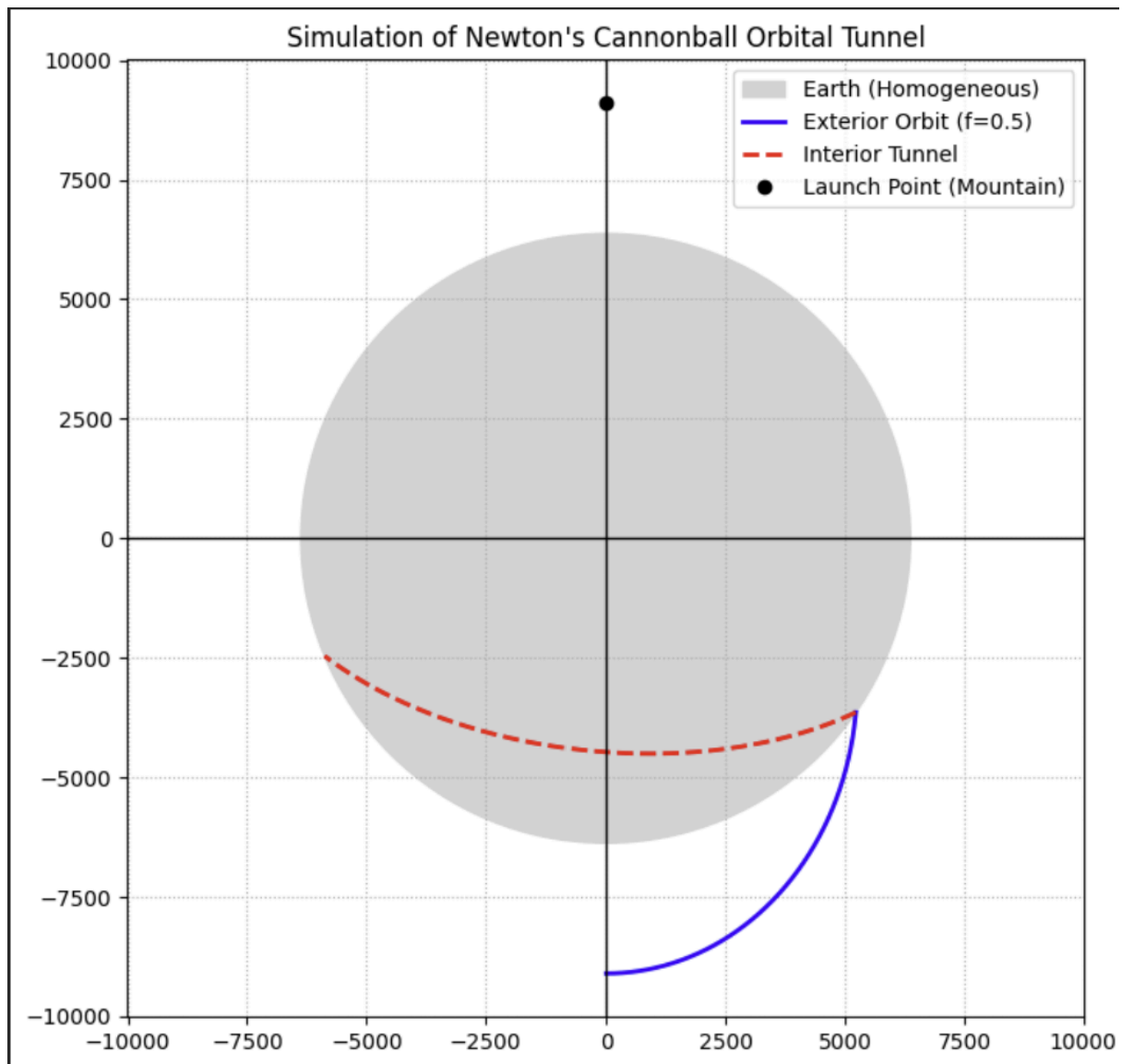


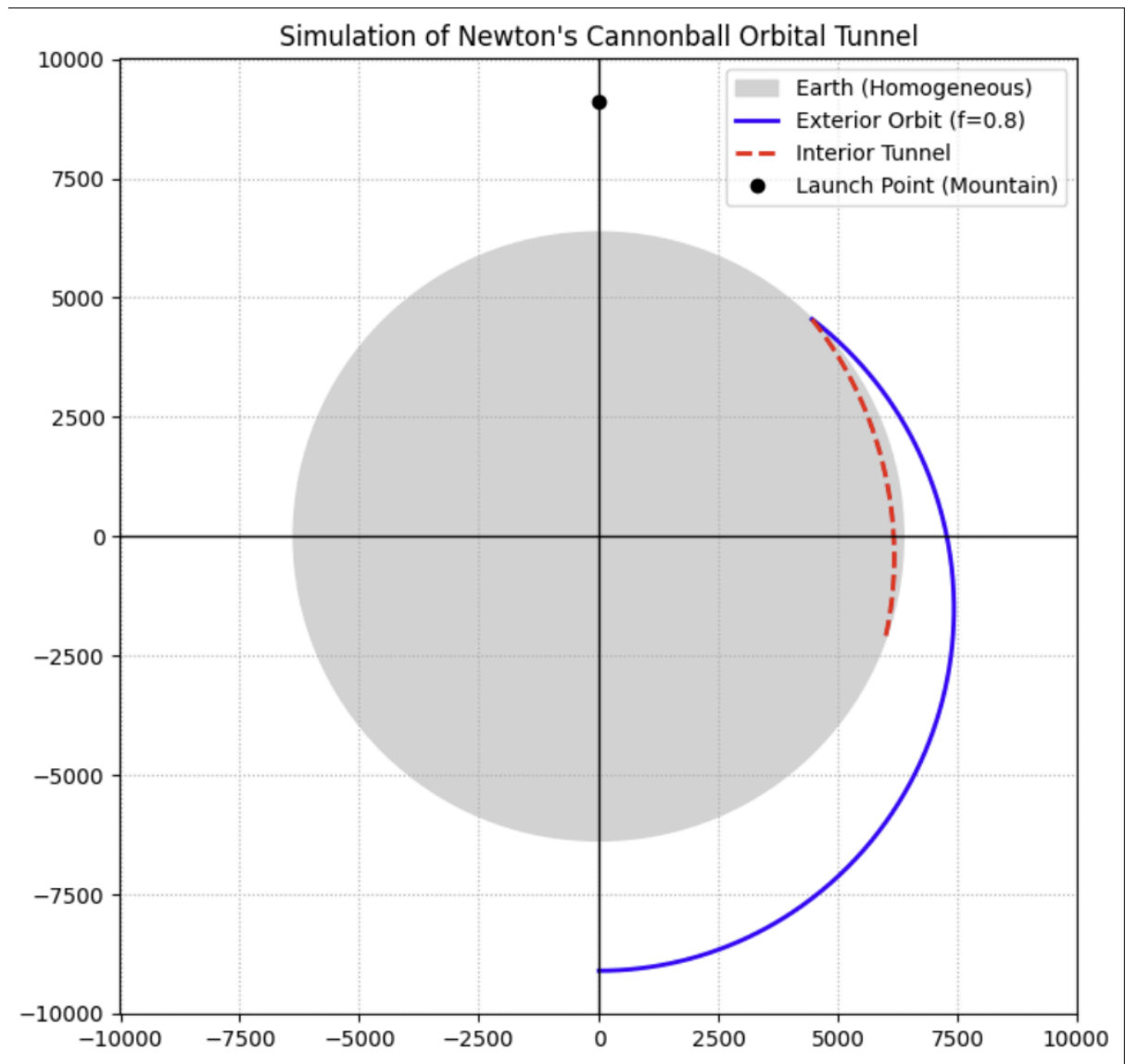
TEAM Project experiment (simulation)

team 18

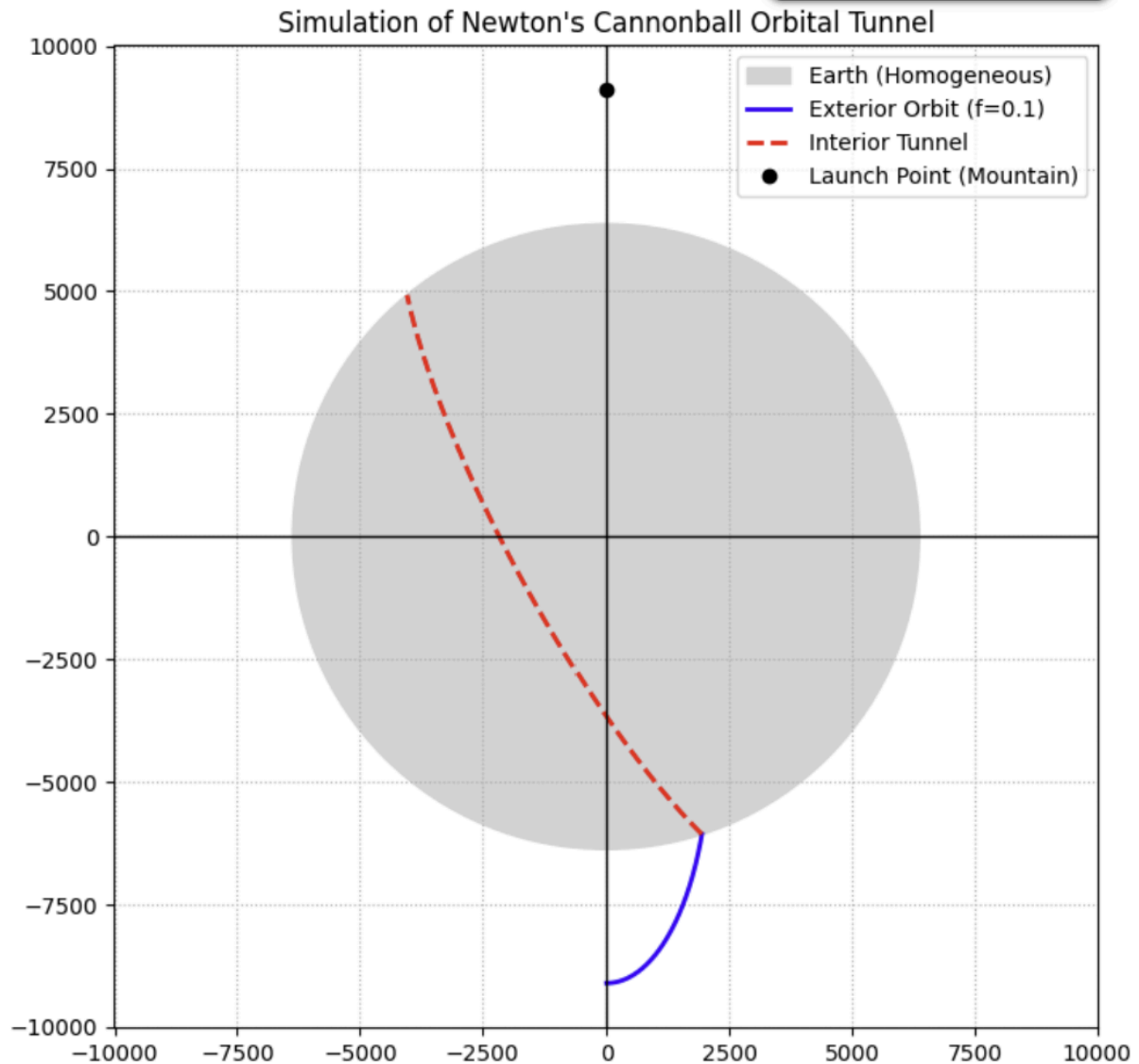
$F=0.5$ normal force situation.



$F=0.8$ Force is almost max



$F=0.1$ small force situation.



CODE

```
import numpy as np
import matplotlib.pyplot as plt

# 1. System Constants and Parameter Setup [1, 2]
R_earth = 6371.0 # Radius of the Earth (km) [2]
a = 0.7          # Normalized radius factor (R/rA), related to mountain height [1, 2]
f = 0.1          # Muzzle velocity ratio (f < f_max) [1, 2]

# rA: Distance from the center of the Earth to the apogee [2, 3]
rA = R_earth / a

# 2. Calculation of Exterior Keplerian Orbit [3, 4]

# Calculate striking (impact) angle theta_0 [2, 5]
cos_theta0 = (f - a) / (a * (1 - f))
theta0 = np.arccos(cos_theta0)
```

```

# External path from launch point (pi) to Earth's surface (theta0)
theta_ext = np.linspace(np.pi, theta0, 500)
r_ext = (rA * f) / (1 + (1 - f) * np.cos(theta_ext))

# 3. Calculation of Interior Quasi-Keplerian Orbit [6]
# Calculate intermediate terms used in interior orbit equations [7, 8]
term1 = 3 - a * (2 - f)
term2 = np.sqrt(term1**2 - 4 * f / a)

# Calculate internal ingress/egress angle phi_s [2, 8]
phi_s = 0.5 * np.arccos((term1 - 2 * f / a) / term2)

# Interior path from ingress point to egress point [8, 9]
phi_int = np.linspace(phi_s, np.pi - phi_s, 500)
x_sq = (2 * f / a) / (term1 - term2 * np.cos(2 * phi_int))
r_int = np.sqrt(x_sq) * R_earth

# 4. Coordinate Transformation for Visualization (Polar -> Cartesian)
x_ext_plot = r_ext * np.sin(theta_ext)
y_ext_plot = r_ext * np.cos(theta_ext)

# Adjust phase of interior path considering physical and geometric symmetry [6, 9, 10]
y_int_plot = r_int * np.cos(phi_int - phi_s + theta0)
x_int_plot = r_int * np.sin(phi_int - phi_s + theta0)

# 5. Plotting the Graph
plt.figure(figsize=(8, 8))

# Represent the Earth (Homogeneous sphere model) [11, 12]
earth = plt.Circle((0, 0), R_earth, color='lightgray', label='Earth (Homogeneous)')
plt.gca().add_patch(earth)

# Trajectory Plot
plt.plot(x_ext_plot, y_ext_plot, 'b-', linewidth=2, label=f'Exterior Orbit (f={f})')
plt.plot(x_int_plot, y_int_plot, 'r--', linewidth=2, label='Interior Tunnel')
plt.plot(0, rA, 'ko', label='Launch Point (Mountain)')

# Set graph formatting
plt.axhline(0, color='black', lw=1)
plt.axvline(0, color='black', lw=1)
plt.xlim(-rA*1.1, rA*1.1)
plt.ylim(-rA*1.1, rA*1.1)
plt.gca().set_aspect('equal')
plt.title("Simulation of Newton's Cannonball Orbital Tunnel")
plt.legend()
plt.grid(True, linestyle=':')
plt.show()

```

